

Overview

Copyright and License

Copyright (c) 2024 by Omnesys Technologies, Inc. All rights reserved.

Warning : This Software Product is protected by copyright law and international treaties. Unauthorized use, reproduction or distribution of this Software Product (including its documentation), or any portion of it, may result in severe civil and criminal penalties, and will be prosecuted to the maximum extent possible under the law.

Omnesys Technologies, Inc. will compensate individuals providing admissible evidence of any unauthorized use, reproduction, distribution or redistribution of this Software Product by any person, company or organization.

This Software Product is licensed strictly in accordance with a separate Software System License Agreement, granted by Omnesys Technologies, Inc., which contains restrictions on use, reverse engineering, disclosure, confidentiality and other matters.

Introduction

The Rithmic API (R | API) provides an environment designed to facilitate high frequency trading. The possible audiences for this facility include traders targeting these strategies:

- Statistical arbitrage strategies
- Program traders executing blocks or portfolio baskets
- Market makers and other market neutral strategies
- Proprietary high frequency single security or multi security strategies

All these strategies are distinguished in that they require seamless access to multiple markets via a normalized API. We provide such an API, with the additional goals of minimizing latency and providing extreme simplicity. The vast majorities of traders near ± 2 sigma require access to the simplest facilities - quotes/prints and limit/market orders. This API was designed for these traders in mind.

The R | API is an interface to the services provided by a market data and trading infrastructure. This infrastructure resides between the R | API and the exchange market data and trade routing interfaces. The infrastructure implements fault tolerance, normalization, state recovery and load balancing allowing client applications to focus on trading.

The R | API provides functionality in the following broad categories:

- **Market Data:** Market data includes trades, quotes and limit order books. Some reference data is available, such as minimum tick increments.
- **Order Placement:** Ability to place/cancel/modify limit, market, stop limit and stop market orders.
- **State Recovery Semantics:** Ability to query state of all open orders, positions and audit trails, so the trading application does not have to implement persistent state semantics for startup or recovery.
- **Alerts and Control Messages:** Alerts and control messages provide status information to the trading application regarding various trading, infrastructure and exchange connectivity. These messages programmatically propagate alerts to the application in case of failures or special conditions as far as possible.

The R | API was designed with specific technical/architectural aims. Its design was driven by our past experience in the industry as well as feedback from traders. The design goals are as follows:

- **Minimizing Latency:** This is the primary design goal. We define system latency to be the period a message (either a market data message or a trade message) is within the infrastructure.
- **Maximizing Throughput:** The design goal also aims to maximize throughput to the client application, with a focus on market data.
- **Transparency/Normalization of Exchange Market Data Protocols:** Each exchange propagates its market data or order information in different ways. The R | API insulates users from these native protocols. Furthermore, changes at the exchange-specific layer can be absorbed by the infrastructure or the R | API where appropriate.
 - **Exchange Market Data Formats:** The market data formats vary from exchange to exchange. The various data formats are normalized into a single format by the R | API and so relieves the user's application of data parsing duties.
 - **Exchange Order Placement:** Various exchanges use different labels for the same trades. These are normalized and translated into simple order types which the client accesses via the R | API.
 - **Exchange Trading Gateways, Recovery and Fault Tolerance:** Every exchange implements failover and recovery semantics on its market data and trading gateways. These semantics also vary from exchange to exchange. The R | API insulates the user from these implementation details and failovers are transparent to the user.
 - **State Recovery:** State recovery is an extremely important aspect of a trading application. The R | API and the infrastructure aim to relieve the trading application, as much as possible, of bookkeeping and state recovery logic. They do this by providing calls to query open positions and open orders from the application. Also provided are some aspects of order and execution history. On startup or recovery, an application can simply query for this information instead of initiating state recovery from a persistent store. This also relieves an application of check pointing duties.
- **Simplicity and Consistency:** The interface is designed for simplicity with consistent interface conventions (such as the naming of properties and methods, and the use of data types).

The R | API makes use of the [zlib](#) library and the [OpenSSL](#) library.